Cheatsheet · 3 chapters on one page

Git and CI/CD cheatsheet

Every command, key idea, table, memory aid and common mistake from the chapters, gathered in one place for quick revision. Nothing here is new. Each card links back to the lesson that teaches it, so open the lesson when a line does not make sense yet.

15 key ideas | 22 command cards | 7 tables | 8 tips and traps

STEP 1 Getting started with Git

1.1 What Git is and how to set it up

Why version control exists, centralised versus distributed, snapshots, installing Git, git config levels and a first repository.

KEY IDEAS

- **Version control** Records every version of your files with author, time and message, so you can compare, restore and work in parallel.
- **Distributed** Every Git clone has the full history. Commit, log and diff work offline. Centralised tools need the server.
- **Snapshots** A commit is a snapshot of the whole project. Unchanged files are reused, not stored again.
- **Config levels** system, global (~/.gitconfig), local (.git/config). Later wins. Set user.name, user.email and init.defaultBranch first.
- **Git is not GitHub** Git is the tool. GitHub, GitLab and Bitbucket host repositories and add pull requests, reviews and CI/CD.

COMMANDS

Set a value for your user — Exam favourite

```
git config --global <key> "<value>"
```

Stored in ~/.gitconfig. Use --local (the default inside a repository) to set it for one repository only.

Lesson: Install Git and tell it who you are

Which value wins — Exam favourite

system < global < local

Git reads /etc/gitconfig, then ~/.gitconfig, then .git/config. A later file overrides an earlier one, so the most specific setting wins.

Lesson: Install Git and tell it who you are

Get help

`git help <command>` or `git <command> -h`

The first opens the full manual page, the second prints a short list of options in the terminal.

Lesson: Install Git and tell it who you are

TABLES

Table: problem without version control · what version control gives you

Problem without version control	What version control gives you
Many copies with confusing names	One folder, a clean history of saved snapshots
No idea what changed	An exact list of changed lines between any two versions
A mistake ruins the only copy	Go back to any earlier version
Two people overwrite each other	Separate branches that are merged with checks
Nobody knows why a line exists	Every change carries an author, a date and a message

Lesson: Why version control exists

Table: kind · examples · where the history lives · weak spot

Kind	Examples	Where the history lives	Weak spot
Local	RCS	One computer	No teamwork, one disk failure loses everything
Centralised	CVS, Subversion (SVN), Perforce	One central server	Server down means nobody can commit or read history
Distributed	Git, Mercurial	Every clone has the full history	More ideas to learn at the start

Lesson: Where Git came from and how it is different

Table: system · how to install

System	How to install
Windows	Download Git for Windows from git-scm.com. It includes Git Bash, a terminal where every command on these pages works.
macOS	Run `git --version`. If Git is missing, macOS offers to install the Xcode command line tools. Homebrew users can run `brew install git`.
Linux (Debian, Ubuntu)	`sudo apt install git`
Linux (Fedora)	`sudo dnf install git`

Lesson: Install Git and tell it who you are

TIPS AND TRAPS

REMEMBER init.defaultBranch decides the name of the first branch in new repositories. Git 2.28 added this setting. If it is not set, current Git still uses master and prints a hint, and the Git project has said that new repositories will use main by default in Git 3.0.

Lesson: Install Git and tell it who you are

REMEMBER Using a work laptop for personal projects? Set your personal email globally and your work email with git config user.email inside each work repository. The local value wins there.

Lesson: Install Git and tell it who you are

COMMON MISTAKE Running git init in the wrong folder, for example your whole home folder, makes Git track everything. If that happens, delete the new .git folder in that place. Your files are not touched.

Lesson: Your first repository in two minutes

IN THE EXAM Git and GitHub are not the same thing. Git is the tool on your computer. GitHub, GitLab and Bitbucket are websites that host Git repositories and add pull requests, issues and CI/CD pipelines.

Lesson: From intuition to research

1.2 The three areas: working tree, staging area and repository

Working tree, staging area and repository, file states and status codes, add, add -p, commit and good commit messages.

KEY IDEAS

- **Three areas** Working tree (your files), staging area or index (next snapshot), repository (all commits). add moves changes to the index, commit saves the index.
- **Status codes** git status -s prints XY: X staged (index vs HEAD), Y unstaged (folder vs index). ?? untracked, MM staged and changed again.
- **Commit saves the index** Edits made after git add are not in the commit until you add again. commit -a stages tracked files only.
- **Partial staging** git add -p stages hunk by hunk so one file can feed two clean commits.
- **Good messages** Short imperative summary near 50 characters, blank line, then why. One logical change per commit.

COMMANDS

Short status — Exam favourite

```
git status -s → XY path
```

X compares the index with HEAD (what is staged). Y compares the working tree with the index (what is not staged yet). ?? means untracked.

Lesson: File states and git status

Long status

```
git status
```

Lists staged changes, unstaged changes and untracked files, with the command to undo each.

Lesson: File states and git status

Stage — Exam favourite

```
git add <file>, git add ., git add -p
```

git add . stages everything under the current folder. -p asks about each changed piece (hunk) so you can stage part of a file.

Lesson: Adding and committing well

Commit — Exam favourite

```
git commit -m "message"
```

Saves the staging area. Without -m Git opens your editor for a longer message.

Lesson: Adding and committing well

Stage tracked files and commit

```
git commit -am "message"
```

-a stages every change to tracked files first. New untracked files are not included.

Lesson: Adding and committing well

Remove and rename

```
git rm <file>, git mv <old> <new>
```

Changes the folder and stages the change in one step. git rm --cached keeps the file on disk but stops tracking it.

Lesson: Adding and committing well

TABLES

Table: area · also called · what it holds · studio picture

Area	Also called	What it holds	Studio picture
Working tree	working directory	The files you see and edit in your folder	the studio floor
Staging area	the index	The exact snapshot your next commit will save	the stage
Repository	.git directory, HEAD	Every commit ever made	the album

Lesson: The three areas of a Git project

Table: state · meaning · how it gets there · short code

State	Meaning	How it gets there	Short code
Untracked	New file Git is not watching	You create it	`??`
Unmodified	Same as the last commit	After a commit or checkout	(not listed)
Modified	Changed, not staged	You edit a tracked file	`M`
Staged	Change is in the index	git add	`M` or `A`
Staged and modified again	Index has one version, the folder a newer one	edit after git add	`MM`

Lesson: File states and git status

Table: rule · good · bad

Rule	Good	Bad
Short summary line, about 50 characters	Fix crash when the password is empty	fixed stuff
Imperative mood, as if giving an order	Add search by price	Added search, adding search
Blank line, then the why in the body	Users saw a 500 error on empty input...	(nothing)
One logical change per commit	One bug fix	Bug fix plus renaming plus new feature

Lesson: Adding and committing well

TIPS AND TRAPS

COMMON MISTAKE git commit saves what is staged, not what is in your file. If you edit a file after git add, the newer edit is not in the commit until you add it again.

Lesson: The three areas of a Git project

REMEMBER Conventional Commits is a popular format: feat: add search by price, fix: handle empty password, docs: update README. Tools read it to make changelogs and choose the next version number.

Lesson: Adding and committing well

1.3 Reading history: log, diff, blame and naming commits

git log filters and the pickaxe, the three diffs and hunk headers, ~, ^, reflog names, .. and ... ranges, and git blame.

KEY IDEAS

- **git log** --oneline --graph --all for the picture. -S text to find when code appeared. -p and --stat for what changed. -- path for one file.
- **Three diffs** git diff: folder vs index. git diff --staged: index vs HEAD. git diff HEAD: folder vs HEAD.
- **Hunks** @@ -a,b +c,d @@: old lines a to a+b-1 became new lines c to c+d-1. A changed line is one - and one +.
- **Naming commits** X~n: n first parents back. X^n: n-th parent of a merge. HEAD@{n}: where HEAD was n moves ago.
- **Ranges** A..B: in B not in A. A...B: in one but not both (log), or merge base to B (diff).

COMMANDS

Compact history Exam favourite

```
git log --oneline --graph --
    decorate --all
```

One line per commit, a text drawing of branches and merges, branch and tag names, every branch not just the current one.

Lesson: Reading history with git log

Filter

```
git log -n 5, --author=Asha, --
since="2 weeks ago", --grep=login
```

Last 5 commits, by one person, in a time window, or with a word in the message.

Lesson: Reading history with git log

What a change touched

`git log -p`, `git log --stat`, `git log -- path/file`

-p adds the full diff, --stat lists changed files with counts, and -- path limits history to one file.

Lesson: Reading history with git log

Find code

Exam favourite

`git log -S "price_limit"`

The pickaxe: commits that added or removed that text. Great for "when did this function appear?".

Lesson: Reading history with git log

One commit in full

`git show <commit>`

Message plus diff. git show HEAD:app.py prints the file as it was in that commit.

Lesson: Reading history with git log

Hunk header

Exam favourite

`@@ -a,b +c,d @@`

The old file from line a for b lines became the new file from line c for d lines. Lines start with space (context), - (removed) or + (added).

Lesson: git diff: three comparisons and how to read a hunk

Word level and stats

`git diff --word-diff`, `git diff --stat`

Shows changed words inside a line, or only counts per file.

Lesson: git diff: three comparisons and how to read a hunk

Ancestors

Exam favourite

`X~n` = n first parents back. `X^n` = the n-th parent of X

HEAD~1 is the parent, HEAD~2 the grandparent. For a merge commit, X^1 is the branch you were on and X^2 is the branch you merged in.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Two dots

Exam favourite

`A..B` = reachable from B but not from A

git log main..feature lists what feature has that main does not.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Three dots

`A...B` = reachable from either but not both

With git log --left-right each commit is marked < or > for its side.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Where HEAD was

`HEAD@{n}`

The n-th previous position of HEAD, from the reflog. HEAD@{1} is where you were before the last move.

Lesson: Naming commits: HEAD, ~, ^ and ranges

Line by line authors

`git blame -L 10,20 app.py`

For each line: the commit, author and date that last changed it. -w ignores whitespace changes, -C finds lines moved from other files.

Lesson: Who wrote this line and why

Ignore big reformat commits

`git blame --ignore-revs-file .git-blame-ignore-revs`

Skip commits listed in the file, such as a project wide formatting run, so blame shows the real author. GitHub reads this file too.

Lesson: Who wrote this line and why

TABLES

Table: command · compares · answers

Command	Compares	Answers
`git diff`	working tree with the index	What have I changed but not staged?
`git diff --staged`	index with HEAD	What will my next commit contain?
`git diff HEAD`	working tree with HEAD	Everything since the last commit
`git diff main feature`	two commits	How do these branches differ?
`git diff main...feature`	merge base of the two with feature	What did feature add since it split off?

Lesson: git diff: three comparisons and how to read a hunk

TIPS AND TRAPS

COMMON MISTAKE git diff with no arguments shows nothing after git add. The changes did not vanish: they moved to the index. Use git diff --staged.

Lesson: git diff: three comparisons and how to read a hunk

IN THE EXAM HEAD~2 and HEAD^^ name the same commit. HEAD^2 is different: it is the second parent of HEAD and only exists if HEAD is a merge commit.

Lesson: Naming commits: HEAD, ~, ^ and ranges